

Royalty Calculation App Documentation

Summary

The Royalty Calculation app is a Retool-based financial management tool designed to automate, track, and report royalty payments and adjustments for licensors. It manages the lifecycle of royalty billing, including the calculation of Minimum Guarantee (MG) balances, processing of sales data, and the generation of monthly royalty reports.

Purpose and Workflow

The primary purpose of the app is to calculate royalty obligations by reconciling sales data against contract terms and MG balances.

Workflow

1. **Data Ingestion:** Sales data is queried and displayed in the "Sales" tab.
2. **Calculation:** The app calculates MG balances by comparing current royalty due and adjustments against prior balances.
3. **Billing:** Users can generate "Royalty Billing Records" for specific periods.
4. **Reporting:** The app generates Excel reports for specific licensors and contracts, incorporating royalty adjustments and sales data.
5. **Closing:** Users can "Close" a month, which locks the data for that period.

Major Data Flows

- **Sales Data:** Fetched via `get_sales` and displayed in the "Sales" table.
- **Royalty Adjustments:** Managed through the "Royalty Adjustment" tab, allowing users to create, update, and view adjustments.
- **MG Balance:** Calculated via `calculateMGBalance`, which aggregates data from `v_all_transactions` and `v_all_royalty_adjustment`.
- **Reporting:** The `create_report` JavaScript block uses `ExcelJS` to generate and download reports based on filtered data.

Technical Components

Key SQL Queries

- `calc_current_mg_balance`: A complex SQL query that calculates the MG balance by subtracting prior royalties and adjustments from the beginning balance.
- `create_changed_records` / `create_open_original_records`: These queries generate the billing records based on whether the month is closed or open.
- `get_sales_records`: Aggregates sales data, handling logic for licensor changes via `target_game_titles`.

JavaScript Logic

- `calculateMGBalance`: The core engine for the app. It orchestrates the retrieval of royalty data and calculates the MG balance for each contract.
- `create_report`: A complex JS block that uses the `ExcelJS` library to format and export data into an Excel file. It includes helper functions for styling, borders, and layout management.
- `transfer_licensors`: Handles the logic for transferring contracts between licensors by inserting new contract records and updating the `active_contract` table.

Key Widgets

- `tabbedContainer1`: The main navigation interface for the app.
- `royalty_table`: Displays the main royalty data.
- `adjustment`: A table widget for managing royalty adjustments.
- `create_sales_modal` / `update_sales_modal`: Modals used for CRUD operations on sales data.

Database Schema (Inferred)

Based on the SQL queries, the following tables are utilized:

- `licensor`: `licensor_id`, `licensor_name`
- `contract`: `contract_id`, `licensor_id`, `mg_beginning_balance`, `pp_beginning_balance`, `contract_name`, `effective_start_date`

- **sales:** child_item_id, order_date, sold_quantity, sales, manufactured_quantity, discount, version_id
- **royalty_adjustment:** child_item_id, adjustment_date, royalty_adjustment, adjustment_note, version_id
- **monthly_royalty:** Stores calculated monthly royalty data.
- **monthly_royalty_status:** Tracks whether a month is closed (status = 1).
- **account:** Stores GL account information.

Appendix: Inventory

Component	Name	Responsibility
Query	calc_current_mg_balance	Calculates MG balances for contracts.
Query	get_sales	Fetches sales data for a date range.
Query	insert_monthly_royalty	Performs bulk insert of monthly royalty records.
JS Block	calculateMGBalance	Orchestrates royalty and MG balance logic.
JS Block	create_report	Generates Excel reports using ExcelJS.
Widget	tabbedContainer1	Main app navigation.
Widget	royalty_table	Displays primary royalty data.

Risks and Open Questions

- **Data Integrity:** The transfer_licensors query uses set foreign_key_checks = 0. This is a high-risk operation that could lead to orphaned records if not managed carefully.
- **Complexity:** The calc_current_mg_balance query is extremely long and contains multiple subqueries. This makes it difficult to maintain and debug.

- **Performance:** The `create_report` function performs multiple asynchronous calls within a loop. Depending on the number of months selected, this could lead to browser performance issues or timeouts.
- **Ambiguity:** The `sales` table schema is inferred from multiple joins; the exact relationship between `parent_item`, `child_item`, and `sales` is complex and relies on effective date ranges.